

JB-Solver

Slogan: Problem Solved.

A financial management assistant that helps users manage personal and shared expenses through automated bill splitting, group expense tracking, repayment reminders, and spending analytics.

1. Project Overview

JB-Solver is a web-based financial management platform designed to simplify expense tracking and bill sharing among individuals and groups.

Our target audience: People who uses money (age 10-100)

The application allows users to:

- Track personal expenses
 - Split bills automatically
 - Monitor spending habits
 - Track debts and repayments
 - Manage group expenses during trips and events
 - Receive reminders for unpaid balances
-

2. Our Motivation

Managing expenses in daily life can be inconvenient and time-consuming.

Common scenarios include:

1. Splitting restaurant bills among friends
2. Tracking shared travel expenses
3. Remembering outstanding debts
4. Monitoring personal spending habits

Most people rely on manual calculations or spreadsheets, which can lead to:

1. Human errors
2. Forgotten repayments
3. Budget overruns
4. Lack of financial awareness

JB-Solver aims to automate these processes and improve financial accountability.

3. Problem Statement

Users currently face several challenges:

Expense Splitting	After meals or group activities, calculating each person's share manually can be tedious
Debt Tracking	People often forgot who owes whome money
Spending Awareness	Many users lack visibility into their spending patterns
Budget Control	Users may exceed budgets without realizing it

All people from different ages have experience at least one of these problems

4. Solution Overview

JB-Solver provides:

Solution features	Problem solved
Automated bill splitting	Expense splitting
Group expense management	Expense splitting + group spending awareness
Repayment tracking	Debt tracking
Spending analytics dashboard to track spending over time	Spending awareness
Budget monitoring	Budget control
Secure user authentication	PDPA

5. Value Proposition

- 1) **Automated Accountability:** Tracks debts and repayments automatically.
 - 2) **Financial Transparency:** Clear visibility of shared expenses.
 - 3) **Convenience:** Eliminates repetitive manual calculations.
 - 4) **Better Financial Habits:** Provides spending insights and budget monitoring.
-

6. User Stories

User Authentication: As a user, I want to create an account and securely log in so that my financial records are protected.

Bill Splitting: As a user, I want to split bills automatically so that I do not need to calculate each person's share manually.

Expense Tracking: As a user, I want to record expenses so that I can monitor my spending and save sufficiently

Group Expenses: As a user, I want to create expense groups so that shared expenses can be managed collectively.

Debt Tracking: As a user, I want to track repayments so that I know who owes me money.

7. Features in detail

1. User Authentication

- User Registration (create account)
- User Login (with the registered account)
- Password Hashing and checking using bcrypt
- JWT Authentication

- PostgreSQL User Storage
-

7.1. Planned Core Features

2. Transaction Management

Record, update, view, and delete financial transactions

Provide full CRUD functionality for expense records

Each transaction contains:

- Amount in specific currency
- Category (e.g food, travel, services...)
- Description
- Date
- Payment method (e.g cash, paynow, paylah...)
- Associated Group

Stored in a database

3. Smart Bill Splitting

Automatic expense distribution using different algorithms based on different bill splitting scenarios:

THREE MAIN AIGORITHMS:

Equal Split (“Going Dutch”)

- Example: \$100 bill shared by 4 people
- Result: \$25 each

Proportional Split (“Pay what you ate”)

- Example: Person A consumes 60% and Person B consumes 40%
- Result: A pays \$60 and B pays \$40

Customised Split

- Example: 3 friends shared fries and each ordered their own main course
- Result: Apply equal split and proportional split

How it works:

Bill Splitting algorithm approach:

- Assign each friend to a variable
- Assign who paid and who is paying (Segmentation)
- Apply algorithms

2 different scenarios of bill splitting

1. One person pays for all
 - Example: \$100 bill, Person A pays for all
2. Shared payment
 - Example: Bill \$300 (michelin meal): Person A only pays \$180 (not enough money), Person B or maybe even more people help to pay

Bill Splitting Examples

Using going dutch method	Using proportional split
Bill \$300 Person A pays \$180 Person B pays \$100 Person C pays \$20 Person D did nothing Person E did nothing Round 1 (minus amount for those who paid): Person A: -180 Person B: -100 Person C: -20 Person D: 0 Person E: 0 Round 2 (Split based on bill splitting method): $\\$300 / 5 = \\60 each (Apply going dutch algorithm) Round 3 (plus amount needed to pay): Person A: $-180 + 60 = -120$ Person B: $-100 + 60 = -40$ Person C: $-20 + 60 = 40$	Bill \$300 Person A pays \$180 Person B pays \$100 Person C pays \$20 Person D did nothing Person E did nothing Round 1 (minus amount for those who paid): Person A: -180 Person B: -100 Person C: -20 Person D: 0 Person E: 0 Round 2 (Split based on bill splitting method): Person A ordered \$70 Person B ordered \$60 Person C ordered \$80 Person D ordered \$50 Person E ordered \$40

Person D: $0 + 60 = 60$ Person E: $0 + 60 = 60$ Round 4: Split into 2 groups Group X: Those with negative amount don't Group Y: Those with positive amount have to pay Loop through Y and X Result (like looping and filling up a bucket until it is full): C pays \$40 to A D pays \$60 to A E pays \$20 to A E pays \$40 to B	Round 3 (plus amount needed to pay): Person A: $-180 + 70 = -110$ Person B: $-100 + 60 = -40$ Person C: $-20 + 80 = 60$ Person D: $0 + 50 = 50$ Person E: $0 + 40 = 40$ Round 4: Split into 2 groups Group X: Those with negative amount don't Group Y: Those with positive amount have to pay Loop through Y and X Result (like looping and filling up a bucket until it is full): C pays 60 to A D pays 50 to A E pays 40 to B
--	--

Customised split
Bill \$300 Person A pays \$180 Person B pays \$100 Person C pays \$20 Person D did nothing Person E did nothing Round 1 (minus amount for those who paid): Person A: -180 Person B: -100 Person C: -20 Person D: 0 Person E: 0 Round 2 (Split based on bill splitting method): Person A ordered \$20 Person B ordered \$10 Person C ordered \$40 Person D ordered \$30 Person E ordered \$20

Everybody ordered \$180 (for sharing => must include "share cost" button)

Round 3 (plus amount needed to pay):

Everybody ordered \$180 => each paid $\$180/5 = \36

Person A: $-180 + 20 + 36 = -124$

Person B: $-100 + 10 + 36 = -54$

Person C: $-20 + 40 + 36 = 56$

Person D: $0 + 30 + 36 = 66$

Person E: $0 + 20 + 36 = 56$

Round 4 (create dummy account X):

Split into 2 groups

Group X: Those with negative amount don't

Group Y: Those with positive amount have to pay

Loop through Y

Result (looping and filling up a bucket until it is full):

C pays A \$56

D pays A \$66

E pays A \$2

E pays B \$54

4. Group Expense Tracking

- Add friends into "group group" and track shared expense management.
- Line graph and pie chart to track expenses types over time

Payment Status Tracking

- Paid / Pending statuses.

User Dashboard

- Financial overview and analytics. User profile

7.2. Planned Supplementary Features

Budget Monitoring

Budget limits and warnings. (Green for save and red for above budget)

Spending Analytics

Charts and visual reports.

Debt Reminders

Users can press the “please pay” button to remind and notify users for unpaid debt.

8. Technology Stack

Frontend	Backend	Database	Security	Version Control
HTML CSS Javascript React Axios	Node.js Express.js	PostgreSQL	bcrypt JSON Web Token (JWT)	Git Github

9. Software Development approach

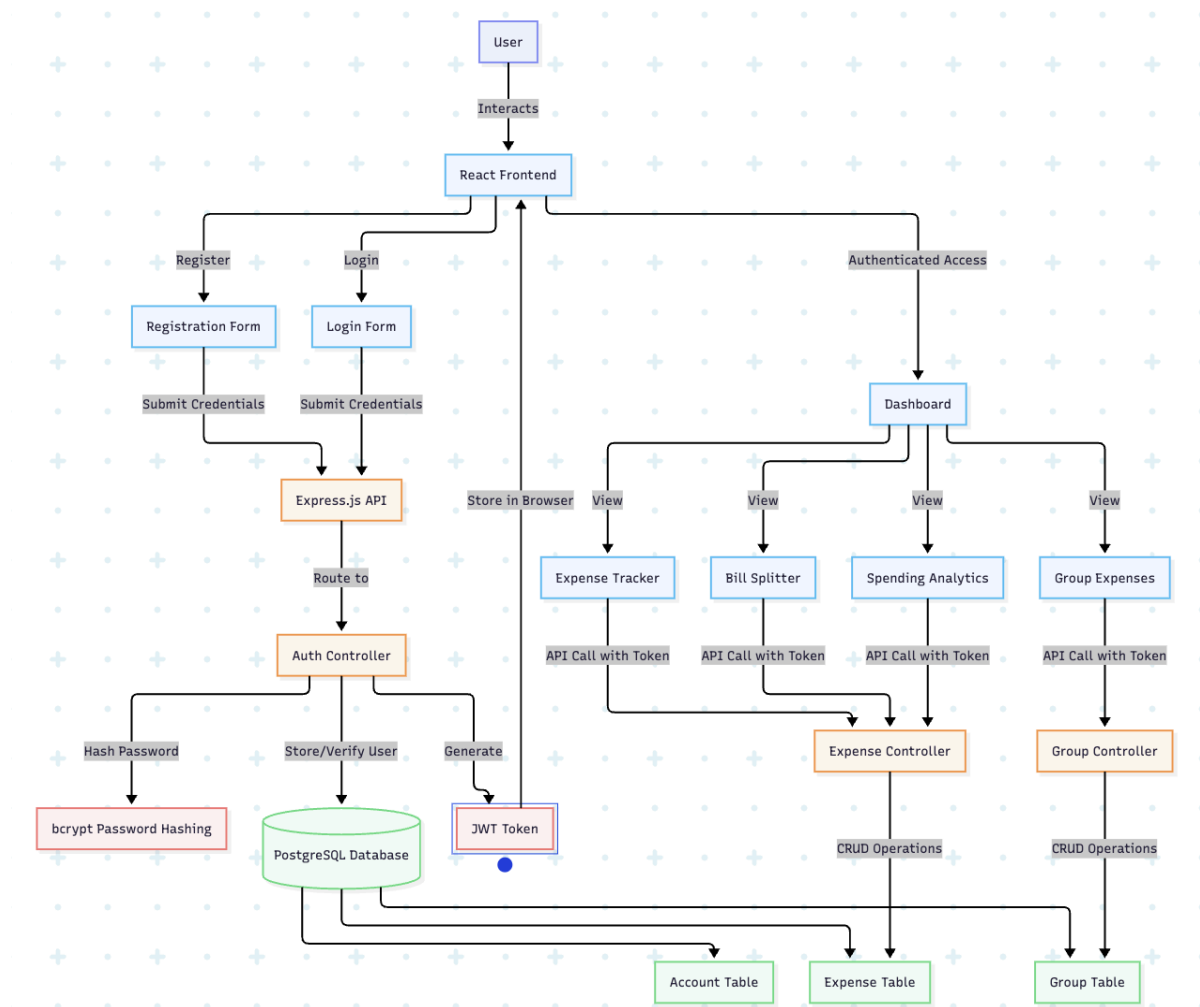
Agile method

- Plan a specific design feature
- Design the feature
- Develop and code out the feature

- Test to see whether frontend, backend and database work
- Feedback
- Repeat the cycle

Strategy: Build one feature at a time

10. Software Architecture (Plan)



11. Timeline and Development plan (Plan)

MS	Tasks	Description	In-Charge	Date
1	Upskilling and Learning	learn how to use the tech stack tools to build the frontend (React, HTML, CSS), backend (Node.js, Express) and database (PostgreSQL), with the help of resource materials like mission control #1, geekforgeeks and coursera to assist us in our learning	BOTH	11th May -24th May
	Create skeleton structure for project folder	Created necessary files and folders, and installed various modules and packages.	BOTH	25th May
	Create postgresql database	Install postgresSQL, pgAdmin 4. Set up server and port: 5433. Created database table and relevant schema (account user)	JONATHAN	26th May
	User authentication and login	Ensuring that users can create and log in to an account. Ensure maintenance of user privacy	BOTH	27th May - 1st June
		Ensure maintenance of user privacy	JONATHAN	27th May - 1st June
		Write backend and frontend logic for user authentication page	BENJAMIN	
	Connect frontend, backend, and database	Connected frontend, backend, and database together.	BOTH	27th May - 1st June
Evaluation Milestone 1 (1st June)				
2	Homepage dashboard	Contains tab for 1) community group, 2) payment tracker, 3) expenditure tracker, 4) User profile	JONATHAN	1st-7th June
		Task-to-do notifications: Render outstanding payments	BENJAMIN	1st-7th June
		Add community group button (also can refer user to community group tab)	BENJAMIN	1st-7th June

	Community group	Users can create/delete community group	JONATHAN	8th-14th June
		Users can add/ delete members into the community group	JONATHAN	8th-14th June
		Contains split-bill functionality and payment tracker	BENJAMIN	8th-14th June
	Payment tracker	Feature payment tracking notifications: Example: Pending/outstanding payments featured like priority list	BENJAMIN	14th-21st June
		Users can add/delete/update payment record	BENJAMIN	14th-21st June
		Payment record must include description, category, date, amount (\$), paid_by_who,	BENJAMIN	14th-21st June
		Feature to tally total amount paid by the entire group (according to payment records)	BENJAMIN	14th-21st June
		Payment status (Pending/Paid) shown for each members in the group	BENJAMIN	14th-21st June
		Users can click on payment transaction button to toggle Payment status to 'Paid'. (Indicate that they have paid)	JONATHAN	14th-21st June
	Split-bill calculation	A clickable button to calculate split-bill	JONATHAN	22nd-28th June
		A split-bill algorithm that takes into account how much each person has to pay. There is also an opt out option in case some members are exempted from paying	JONATHAN	22nd-28th June
		Write codes for possible split-bill scenarios: 1) Equal splitting('Going dutch'), 2) proportional split('pay what you ate'), 3) customised split. Take into account whether 1) one person pays all 2) complicated shared payment	BENJAMIN	22nd-28th June

Evaluation Milestone 2: (29th June)				
3	Expenditure tracker	Data visualization: Show breakdown of user expenditure base on catergories (Pie chart)	JONATHAN	29th June - 5th July
		Show expenditure breakdown by day, weeks and months (Using bar charts and Graphs for comparison)	BENJAMIN	29th June - 5th July
		Feature transaction history (all payment transactions)	BENJAMIN	29th June - 5th July
	User profile	User can update their username, email, and password	JONATHAN	6th-12th July
		User can delete their account or log out	JONATHAN	6th-12th July
		User can set their profile avatar (for personalisation)	JONATHAN	6th-12th July
	Homepage dashboard	Personal expenditure section: Users can add individual payment record (Payment record must include description, caterygory, date, amount (\$)),	BENJAMIN	13th-19th July
		Improve UI/UX of homepage	BENJAMIN	13th-19th July
	Community group	Improve UI/UX of homepage	BENJAMIN	20th-26th July
	Payment tracker	Improve UI/UX of homepage	BENJAMIN	20th-26th July
		Please pay' button to notify/remind other members to pay you money. (Potentially can work on syncing with their email notification)	JONATHAN	20th-26th July
Evaluation Milestone 3: (27th July)				
4	Refinement	Expenditure tracker	JONATHAN	
		Payment tracker: Online Paynow payment sync with email and send notification to JBSolver to show that users have paid	JONATHAN	
		User profile: Improve the functionality of the profile avatar	BENJAMIN	

		Homepage dashboard	BENJAMIN	
Splashdown: (26th August)				

12. Project Structure

backend/

```

├── config/
|   └── db.js ## connect JBSolver to the postgresql database ###
├── controllers/
|   └── userController.js ## handle user-related logic ###
├── routes/
|   └── userRoutes.js ## connects URLs to controllers ##
├── .env ##
└── server.js ## main entry point of backend ##

```

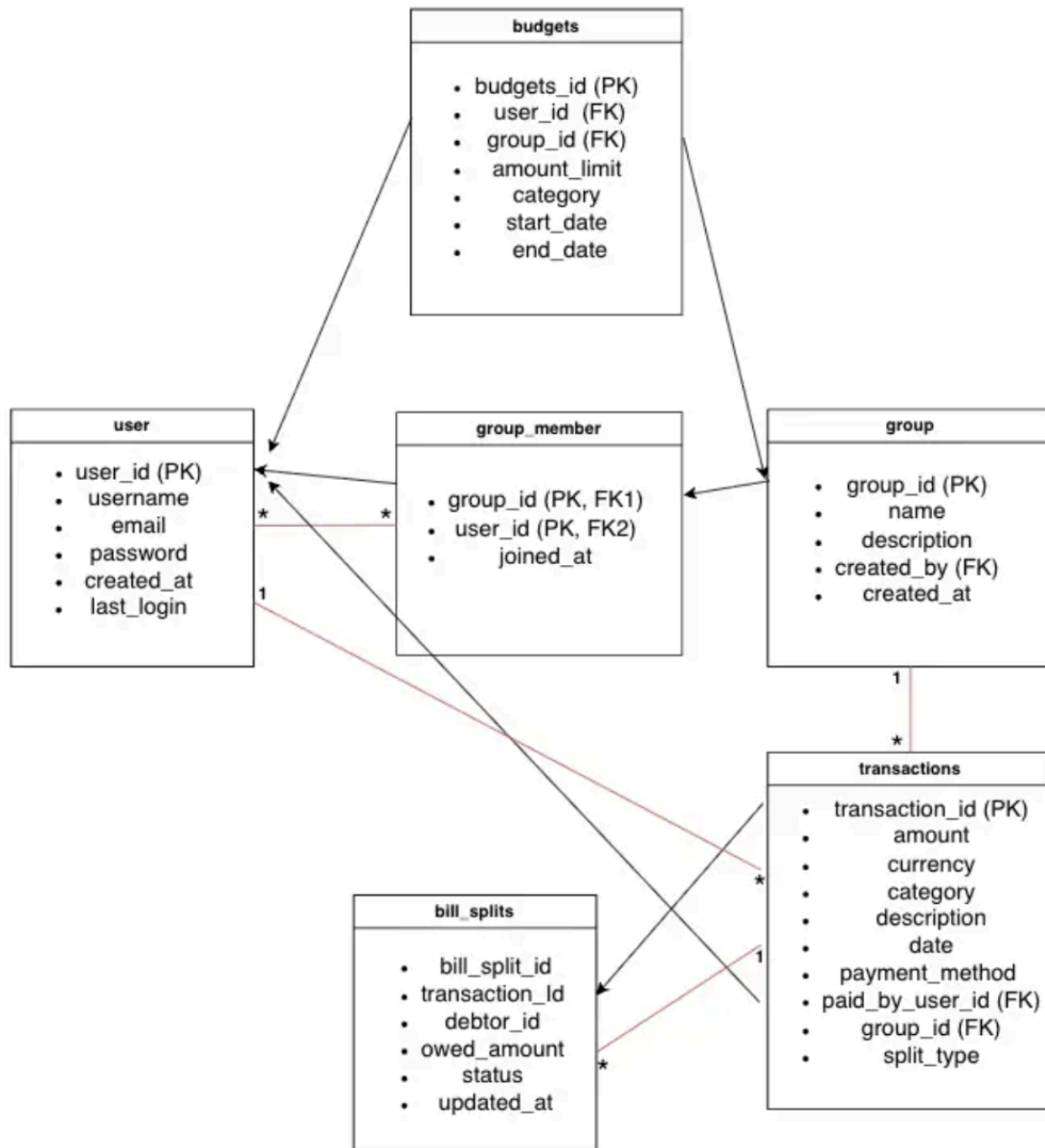
frontend/

```

├── src/
|   ├── App.jsx ## root component for the entire user interface ##
|   ├── App.css
|   └── main.jsx

```

Relational database Schema design



13. Installation Guide and Rendering on Browsers

Backend

cd backend

npm install

node server.js

Frontend

cd frontend

npm install

npm run dev

14. Usage Guide

1. Register a new account.
 2. Login using credentials.
 3. Access protected homepage.
 4. View user profile information.
-

15. Future Enhancements

- Expense categorization
 - AI-powered spending insights
 - OCR receipt scanning
 - Multi-currency support
 - Mobile application
 - Push notifications
-

16. Contributors

Jonathan Wong

Benjamin Wong